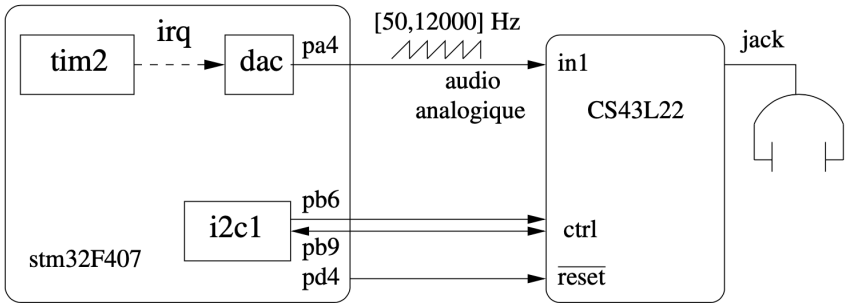
Université Grenoble Alpes - L3 Informatique
2023-2024

Architectures logicielles et matérielles

TP 1 Semaine 5

Ce TP sur carte STM32F4 a pour objet l'utilisation du circuit audio CS43L22. L'objectif sera de générer un son amplifié par ce circuit audio sur la sortie casque. Nous lui transmettrons une suite d'entiers, **la cadence d'émission étant définie par le timer TIM2**. *Mais avant tout, notre programme devra* **configurer le CS43L22 en amplificateur**. *Cette configuration se fera via l'interface I2C.*



Informations préliminaires (à lire attentivement)

• La configuration du CS43L22 s'effectue par l'interface I2C1. Les signaux de cette interface sont connectés aux **ports GPIO** comme suit : SDA (Serial Data) sur port GPIOB, pin 9, et SCL (Serial Clock) sur port GPIOB, pin 6.

• Nous aurons à consulter certains bits dans les **registres** *status* **SR1 et SR2** de l'interface I2C1. La documentation de la carte STM32F4 fournit les schémas ci-dessous de ces deux registres.

Avant d'initier une communication avec le périphérique, nous devrons consulter le bit 1 (BUSY) du registre SR2 pour savoir si le bus I2C est libre pour une communication (bit à 0 s'il n'y a pas de communication sur le bus).

Address offset: 0x18
Reset value: 0x0000

Note: Reading I2C_SR2 after reading I2C_SR1 clears the ADDR flag, even if the ADDR flag was set after reading I2C_SR1. Consequently, I2C_SR2 must be read only when ADDR is found set in I2C_SR1 or when the STOPF bit is cleared.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PEC[7:0]								DUALF	SMB HOST	SMBDE FAULT	GEN CALL	Res.	TRA	BUSY	MSL
r	r	r	r	r	r	r	r	r	r	r	r		r	r	r

Le registre SR1 nous fournira diverses informations sur l'état de la communication, en particulier pour consulter les valeurs des bits d'acquittement. Nous aurons ici à consulter : le bit 0 (SB, start bit : à 1 lorsque la condition de start a été émise), le bit 1 (ADDR, address sent/matched : à 1 lorsque le périphérique a reconnu son adresse), le bit 2 (BTF, byte transfer finished : à 1 lorsqu'un octet de donnée a été reçu), et le bit 6 (RxNE, data register not empty : à 1 lorsque le registre de données n'est pas vide).

Address offset: 0x14
Reset value: 0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SMB ALERT	TIME OUT	Res.	PEC ERR	OVR	AF	ARLO	BERR	TxE	RxNE	Res.	STOPF	ADD10	BTF	ADDR	SB
rc_w0	rc_w0		rc_w0	rc_w0	rc_w0	rc_w0	rc_w0	r	r		r	r	r	r	r

Notons aussi que les macros suivantes sont définies pour les accès aux contenus de SR1 et SR2 de l'interface I2C1 :

```
// #define MMIO32(addr) (*(volatile uint32_t *)(addr))

/* Status register 1 */
#define I2C_SR1(i2c_base) MMIO32((i2c_base) + 0x14)
#define I2C1_SR1   I2C_SR1(I2C1)

/* Status register 2 */
#define I2C_SR2(i2c_base) MMIO32((i2c_base) + 0x18)
#define I2C1_SR2   I2C_SR2(I2C1)
```

• La documentation Cirrus Logic du CS43L22 fournit les caractéristiques de ce périphérique, et donne notamment un descriptif pour **lecture et écriture** dans les registres de contrôle de ce périphérique **via interface I2C** (proches des exemples vus en cours). Les figures ci-dessous en sont extraites.

Une **écriture** dans un registre du CS43L22 s'effectue comme suit :
- émission d'une marque start
- envoi de l'octet formé de l'adresse du CS43L22 (soit 0x4a dans les 7 bits de poids forts) et un bit de poids faible à 0 indiquant une écriture
- envoi du numéro (map, memory address pointer) de registre interne du CS43L22 dans lequel on veut écrire
- envoi du contenu à écrire dans reg[map]
- émission d'une marque de stop.

Après chaque octet, le périphérique répond avec un bit d'acquittement.

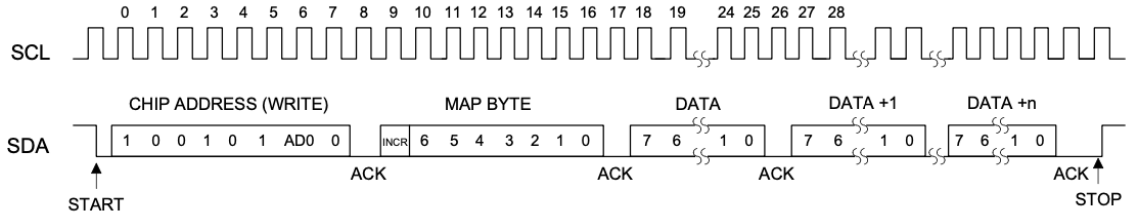
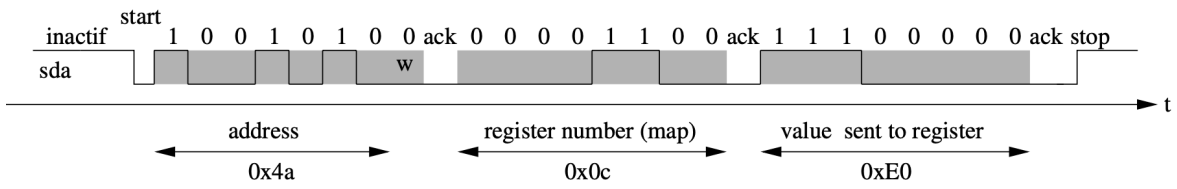


Figure 16. Control Port Timing, I²C Write

La figure ci-dessous illustre par exemple l'écriture de la valeur 0xE0 dans le registre 0xC :



Une **lecture** dans un registre du CS43L22 s'effectue comme suit :
- émission d'une marque start
- envoi de l'octet formé de l'adresse du CS43L22 (soit 0x4a dans les 7 bits de poids forts) et un bit de poids faible à 0 indiquant une écriture
- envoi du numéro (map) de registre interne du CS43L22 dans lequel on veut lire
- nouvelle émission d'une marque start
- envoi de l'octet formé de l'adresse du CS43L22 et un bit de poids faible à 1 indiquant une lecture
- lecture des données
- émission d'une marque de stop.

Le récepteur renvoie un bit d'acquittement pour chaque octet transféré (0 pour ACK, 1 pour NACK - cf cours, et figure).

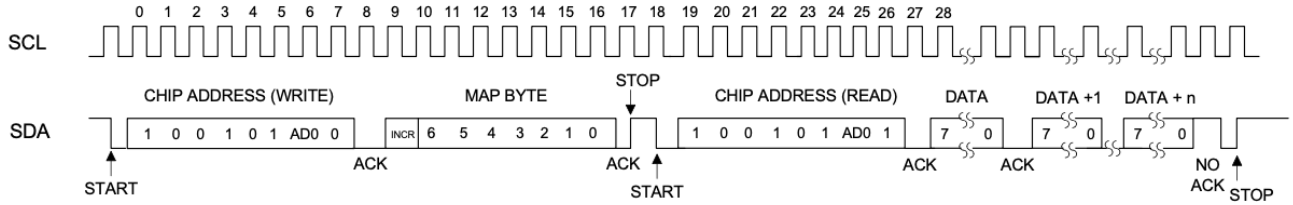
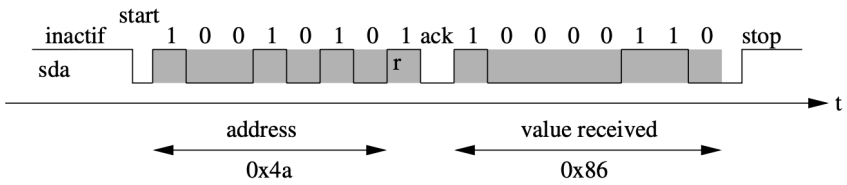


Figure 17. Control Port Timing, I²C Read

La figure ci-dessous illustre par exemple la phase finale de lecture d'une donnée 0x86 :



• Enfin, nous avons indiqué qu'ensuite des entiers seront transmis au périphérique audio selon une période qui sera donnée par le **timer TIM2**. Comme le *systick* que nous avons utilisé précédemment, TIM2 est capable de déclencher une interruption périodiquement, avec un intervalle de temps choisi. Mais comme le TIM4 évoqué en cours, ce timer est extérieur au microcontrôleur, et ses interruptions sont gérées par le NVIC (Nested Vectored Interrupt Controller). Elles doivent être annulées (acquittées), et pour ce faire la bibliothèque *libopencm3* fournit la fonction :

```
void timer_clear_flag(uint32_t timer_peripheral,  // adresse de base périphérique
                      uint32_t flag);
-> Cette fonction annule l'interruption déclenchée par le timer, le deuxième paramètre donne
des flags pour le registre status.
```

Notons que l'adresse de base du TIM2 est simplement fournie par une macro *TIM2* et que l'entier servant de flag ici sera rempli de 1.

Question 1.

Récupérer le fichier audio.tar dans le moodle et placez-le dans le répertoire de votre choix. Désarchiver son contenu. Il contient l'application pour ce TP, principalement située dans les fichiers firmware.c, i2c_audio.c et cs_audio_config.c

Vous constatez que la fonction *main* dans firmware.c effectue les configurations du circuit audio et du timer TIM2 puis réalise une boucle infinie. **Pourquoi cette boucle infinie vide ?** (qui demeurera ainsi) Il est important d'avoir déjà compris cela, compte tenu du descriptif de l'application présenté ci-dessus. Nous reviendrons sur ce point à la question 6.

Dans l'immédiat, remarquez que cette fonction *main* appelle entre autres la fonction *configure_ i2c_ audio()*, définie dans i2c_audio.c. Cette fonction appelle les fonctions *cs_ begin_ init()*, *cs_ config()* et *cs_ end_ init()*, elles-mêmes définies dans cs_audio_config.c. Toutes ces fonctions font appel à la **fonction write_cs()** qui est la fonction utilisée pour **l'écriture dans les registres de contrôle du CS43L22**, via l'interface I2C. C'est l'une des fonctions que vous allez devoir écrire plus loin.

Question 2.

Pour commencer, nous allons écrire dans i2c_audio.c **quelques fonctions de base** qui serviront à consulter l'état de la communication et les bits d'acquittement.

Comme expliqué plus haut, il sera nécessaire de s'assurer que le bus I2C est libre pour une communication (bit BUSY du registre SR2), avant de tenter d'initier une communication. **Ecrire la fonction**

```
void wait_not_busy();
```

qui sert à attendre que ce bit BUSY passe à 0.

Nous avons aussi indiqué que le bit SB du registre permet de savoir si la condition de start a bien été émise. **Ecrire la fonction**

```
void wait_start_sent();
```

qui sert à attendre que ce bit SB passe à 1.

De la même façon, les bits ADDR, BTF, et RxNE servent respectivement à savoir si le périphérique a bien reconnu son adresse, s'il a bien reçu une donnée, et si le master a bien reçu une donnée. **Ecrire les fonctions**

```
void wait_address_sent();
void wait_byte_sent();
void wait_byte_received();
```

qui servent respectivement à attendre que les bits ADDR, BTF, et RxNE passent à 1. Finalement, **écrire la fonction**

```
void clear_addr_bit();
```

qui sert à remettre le bit ADDR de SR1 à 0 (ce qui se fait en consultant le contenu du registre SR2).

Question 3.

Nous allons en venir à la réalisation des fonctions de communication via l'interface I2C. **Les fonctions écrites dans cette question et la suivante auront recours aux fonctions de la question 2**, ainsi qu'aux fonctions suivantes de la *libopencm3* :

```
void i2c_send_start (uint32_t i2c); // adresse de base interface i2c
-> Cette fonction sert à envoyer la condition de start

void i2c_send_stop (uint32_t i2c); // adresse de base interface i2c
-> Cette fonction sert à envoyer la condition de stop

void i2c_send_7bit_address (uint32_t i2c,  // adresse de base interface i2c
                            uint8_t slave, // adresse de l'esclave
                            uint8_t readwrite);
-> Cette fonction sert à envoyer l'adresse 7 bits de l'esclave (le périphérique), suivie
d'un bit indiquant écriture ou lecture (paramètre readwrite). La macro I2C_WRITE pourra être
utilisée ici pour "écriture" et la macro I2C_READ pour "lecture".

void i2c_send_data (uint32_t i2c, // adresse de base interface i2c
                    uint8_t data);
-> Cette fonction sert à envoyer un octet de donnée data

uint8_t i2c_get_data (uint32_t i2c); // adresse de base interface i2c
-> Cette fonction récupère la donnée reçue dans le registre de données

void i2c_disable_ack (uint32_t i2c); // adresse de base interface i2c
-> Cette fonction sert à désactiver l'acquittement
```

Avant toute chose, afin de clarifier le code des fonctions de lecture et écriture, **écrire la fonction**

```
void send_address(bool read);
```

qui va envoyer l'adresse du périphérique CS43L22 ainsi que le mode d'accès (lecture ou écriture) sur le bus I2C.

Ensuite, dans le fichier i2c_audio.c, **compléter le code de la fonction write_cs** pour l'écriture sur l'I2C1. Cette fonction prend en paramètres un numéro de registre du CS43L22 et un octet de donnée, et écrit cette donnée dans le registre. Elle devra successivement :

- Attendre que le bus I2C devienne libre pour une communication
- Emettre une condition de start
- Attendre que le bus soit prêt à l'émission de l'adresse (fin du start)
- Envoyer l'adresse du CS43L22, avec bit d'écriture
- Attendre la fin de l'envoi de l'adresse (le périphérique a bien reconnu son adresse)
- Remettre le bit ADDR de SR1 à 0
- Envoyer le numéro du registre à écrire *regnum*
- Attendre que le périphérique ait reçu
- Envoyer la donnée à écrire *regval*
- Attendre que le périphérique ait reçu
- Emettre une condition de stop

Question 4.

Toujours dans le fichier i2c_audio.c, **compléter le code de la fonction read_cs** pour la lecture sur l'I2C1. Cette fonction prend en paramètre un numéro de registre du CS43L22 et en retourne le contenu. Elle devra successivement :

- Attendre que le bus I2C devienne libre pour une communication
- Emettre une condition de start
- Attendre que le bus soit prêt à l'émission de l'adresse (fin du start)
- Envoyer l'adresse du CS43L22, avec bit d'écriture
- Attendre la fin de l'envoi de l'adresse (le périphérique a bien reconnu son adresse)
- Remettre le bit ADDR de SR1 à 0
- Envoyer le numéro du registre dans lequel lire *regnum*
- Attendre que le périphérique ait reçu
- Emettre une condition de stop
- Attendre que le bus I2C devienne libre pour une communication
- Emettre une condition de start
- Attendre que le bus soit prêt à l'émission de l'adresse (fin du start)
- Envoyer l'adresse du CS43L22, avec bit de lecture
- Attendre la fin de l'envoi de l'adresse (le périphérique a bien reconnu son adresse)
- Remettre le bit ADDR de SR1 à 0
- Désactiver l'acquittement
- Attendre que la donnée soit reçue (registre de données non vide)
- Récupérer la donnée dans le registre de données
- Emettre une condition de stop

Question 5.

Avant d'envisager l'utilisation de ces fonctions, il est essentiel de s'assurer qu'elles sont bien opérationnelles. Pour tester ces deux fonctions, nous allons d'abord tester la lecture en lisant le contenu (connu) d'un registre, puis tester l'écriture en écrivant une valeur dans un registre et en relisant le contenu.

Test de la fonction de lecture : la documentation du CS43L22 indique que le registre ID (à l'adresse 0x01) contient une valeur comprise entre 0xE0 et 0xE3. Placer dans le *main* seulement une lecture du contenu de ce registre et mettre cette valeur dans une variable v.
Compiler et charger l'exécutable.

Placer un breakpoint à la suite de cette ligne de code. Continuer l'exécution jusqu'à ce breakpoint et afficher dans gdb la valeur de v. Vérifier que vous avez bien lu une valeur telle qu'attendu. Si cela est correct, nous allons pouvoir passer au test de l'écriture.

Test de la fonction d'écriture : nous allons utiliser le registre à l'adresse 0x02 pour ce test. La valeur de ce registre à l'origine est 0x01. Placer dans le *main* seulement une écriture de la valeur 0x22 dans ce registre, puis une relecture de la valeur de ce registre.
Compiler et charger l'exécutable.
Placer un breakpoint à la suite de cette ligne de code. Continuer l'exécution jusqu'à ce breakpoint et afficher dans gdb la valeur lue. Vérifier que vous avez bien lu la valeur 0x22 en hexadécimal (34 en décimal).

Si cela est correct, nous pourrons remettre la valeur 0x01 dans ce registre, et finir le développement de notre application.

Question 6.

Comme déjà indiqué, toutes les fonctions pour la configuration du périphérique audio sont déjà fournies, vous n'avez plus rien à modifier de ce point de vue (remettez juste le *main* dans son état d'origine).

Ce circuit audio étant opérationnel, il ne reste plus qu'à lui **envoyer périodiquement** des valeurs entières via le DAC du microcontrôleur, et entendre le son correspondant dans le casque. La fonction *do_cycle_dac()* dans le fichier firmware.c est prévue pour envoyer ainsi une nouvelle valeur, au moyen de la fonction *dac_load_data_buffer_single()* de la *libopencm3*. Vous ne devez rien modifier dans cette fonction.

L'objectif est d'utiliser cette fonction pour envoyer une nouvelle valeur chaque fois que TIM2 (déjà configuré comme il se doit dans le *main*) génère une nouvelle interruption.
Compléter le code en conséquence. Il reste à compiler, charger et tester l'application.

Enfin on pourra noter qu'un appel à la fonction *tim2_change_period(tim2_period)* est présent mais commenté dans le gestionnaire d'interruption *tim2_sr()*. Décommentez cet appel et relancez l'application.
Quelle différence observez-vous et comment l'expliquez-vous ?